

Together in formation

Based on this understanding, the above recursive relations can be written as follows:

For factorial:

```
fact(n) = n * fact(n - 1), for n > 0
fact(n) = 1, for n = 0
```

For Fibonacci:

```
fib(n) = fib(n - 1) + fib(n - 2), for n > 1
fib(n) = 1, for n = 1
fib(n) = 0, for n = 0
```

For exponent:

```
exp(x) = (exp(x/2))^2, for x > 0.001
exp(x) = 1 + x, for x <= 0.001
```

What is this 0.001 for the exponent? Note that x is a real number and if it is positive, dividing it by 2 however many times would never reach absolute zero, though it would become smaller and smaller, approaching zero. Hence, you would never be able to stop, if you put $x > 0$. So, take an approximation, depending on the expected accuracy of the result. And, for this reason, you cannot put $e(x)$ just equal to 1 for $x < 0.001$, but need to have some variability based on x , and hence $1 + x$. Moreover, as it makes sense to have x take all real values (positive, zero, and negative), the conditions could be further enhanced by replacing x by its absolute value. So, for the exponent:

```
exp(x) = (exp(x/2))^2, for abs(x) > 0.001
exp(x) = 1 + x, for abs(x) <= 0.001
```

Together in action

Given that we have these mathematical formulations, putting them down into recursive functions of a programming language is a mere mechanical task. Here we go, all three in a single file, *recursion.bc*:

```
define fact(n) {
  if (n <= 0) # < to avoid -ve value cases
    return 1
  return (n * fact(n - 1))
}
define fib(n) {
  if (n <= 0) # < to avoid -ve value cases
    return 0
  if (n == 1)
    return 1
  return (fib(n - 1) + fib(n - 2))
}
define abs(x) {
```

```
  if (x >= 0)
    return x
  return -x
}
define exp(x) {
  if (abs(x) <= 0.001)
    return (1 + x)
  return (exp(x/2) * exp(x/2))
}
```

The execution and usage is demonstrated here:

```
$ bc -q1 recursion.bc
fact(0)
1
fact(5)
120
fib(1)
1
fib(2)
1
fib(3)
2
fib(4)
3
fib(5)
5
fib(6)
8
exp(0)
1
exp(1) # Shall be an approximation
2.71695572946643553835
e(1)
2.71828182845904523536
quit
```

Summing up

So we have pretty much explored all the functionality of the bench calculator, and if you are a programming geek, you may even go on to try the next level of what's possible—the next dimension, vectors, etc, using arrays of *bc*. Aha! But isn't there a way for non-programmers? Yes, there is Octave.

So, watch this space and stay tuned for an update on

Octave. 

By: Anil Kumar Pugalia

The author is a hobbyist in open source hardware and software, with a passion for mathematics. His exploration with mathematics, in every aspect of life, dates back to the 1990s. A gold medalist from the Indian Institute of Science, mathematics and knowledge-sharing are two of his many passions. Apart from that, he experiments with Linux and embedded systems to share his learnings through his weekend workshops. Learn more about him and his experiments at <http://sysplay.in>. He can be reached at email@sarika-pugs.com.